OR-Select: Reproducible Best-of-N Answer Selection over LLM Ensembles, with an Exact Kemeny Reference and a Ground-Truth Audit

Aris Dressino[a,*]

[a] *TUM School of Management, Technical University of Munich, Arcisstraße 21, Munich, 80333, Germany*

Abstract

Sampling N answers from a language model and choosing one is the standard way to buy accuracy at inference time, but the choosing rule is either majority vote, which fails when answers do not repeat, or an LLM judge whose verdicts are not reproducible. We cast selection as an integer program over a judge-derived pairwise matrix and a deterministic self-consistency term; given the judge's outputs the winner is exactly reproducible (the shipped objective reproduces all 480 replayable stored winners). The Copeland-style aggregate is a 5-approximation to the NP-hard Kemeny-optimal ranking, and we report the exact Kemeny–Young winner as a reference. We audit the method against self-consistency and greedy sampling on GSM8K, MATH Level 5, AIME 2026 and GPQA Diamond (402 problems) with homogeneous and heterogeneous generation, paired exact tests, a headroom/recovery metric and full coverage accounting. The audit is negative. The judge scores 60.5% on RewardBench; with homogeneous generation OR-Select is indistinguishable from self-consistency (7 versus 8 discordant problems of 399), and with heterogeneous generation it is significantly worse (2 versus 25 of 286, $p < 0.001$). The default cost term, measured in words, dominates the quality term in 444 of 480 replayed problems, and removing it does not help because the judge's signal concentrates picks on one generator (69.7%) that is no more accurate. Reproducible selection is not accurate selection; we release the protocol and replayable data.

*Corresponding author. ORCID: 0009-0003-0693-9051.
Email address: aris.dressino@tum.de (Aris Dressino)

Keywords: operations research, large language models, integer programming, answer selection, self-consistency, rank aggregation, benchmarking

1. Introduction

Large language models are sampled, not computed: even greedy decoding is not reproducible across runs or hardware because of GPU batch-invariance and floating-point non-associativity. The usual response is to draw N candidate answers and pick one. Self-consistency [1] picks the majority final answer; it is well-defined only when candidates cluster, and on open-ended queries it degenerates to an arbitrary tie-break (in our open-ended pilot, zero of $N = 5$ candidates matched exactly on every query). Universal self-consistency [2] and judge-based selection replace the vote by an LLM's choice, at the price of a selection step that is itself a stochastic LLM call.

We do not try to make generation deterministic. We treat *selection* over an already-sampled set as an optimization problem: an LLM judge produces a pairwise preference matrix, and an integer program combines it with a deterministic self-consistency signal, a confidence estimate and a cost penalty to pick exactly one winner. Given fixed candidates and fixed judge outputs, the winner is exactly reproducible. What is not automatically reproducible, or even accurate, is the judge; we therefore evaluate the selector against ground truth rather than assume that a principled aggregation rule translates into better answers [3].

Contributions..

1. **Formulation and replayability.** An integer-programming formulation of best-of-N selection that treats self-consistency as one deterministic term next to an LLM-judged pairwise term, together with a storage format that makes every selection exactly replayable offline (Section 3.2).
2. **Reference solutions.** The tractable Copeland-style aggregate inherits the 5-approximation guarantee of Coppersmith et al. [4] against the NP-hard Kemeny-optimal ranking; we additionally compute the exact Kemeny–Young winner on every problem and report how often, and in which direction, the two differ (Sections 3.4 and 5.6). The approximation result is not ours; its empirical consequence for LLM judging is.

3. **An audit protocol and open data.** Paired exact tests against self-consistency, greedy and exact Kemeny–Young; a floor/oracle *recovery* metric that separates selector quality from how much there was to select; valid/total coverage reporting with a flagged, higher-cap second pass for truncated generations; a judge-only validation on RewardBench; and offline weight ablations that cost no API calls because the judge outputs are stored (Sections 4 and 5).
4. **Findings.** (a) The judge is only moderately reliable (60.5% on RewardBench). (b) With homogeneous generation, judge-based selection does not beat majority vote. (c) With heterogeneous generation it is significantly worse, and the mechanism is identifiable: a length-scale mismatch in the default objective plus a generator-specific preference of the judge. (d) The exact Kemeny–Young winner differs from the shipped selector's winner on 91 of 605 problems but is not more accurate.

The negative result is the point of contribution 4: judge-based aggregation is attractive in theory, and the literature on correlated judge errors [5, 6] and on the weak link between judge metrics and selection accuracy [3] predicts trouble; this paper measures it on four benchmarks with an exact reference and explains where it comes from.

2. Related Work

Selection among sampled answers.. Self-consistency [1] takes a majority vote over extracted answers; universal self-consistency [2] lets the model itself choose the most consistent free-form response. Pairwise approaches are closest to ours: PAIRS [7] finds that pairwise comparison plus rank aggregation outperforms asking an evaluator to score or select directly, LLM-Blender [8] ranks the outputs of several LLMs by pairwise comparison before fusing the best ones, and LLM-PeerReview [9] has multiple LLM judges score candidates and aggregates with either averaging or a Dawid–Skene-style graphical truth-inference model. We aggregate instead with an integer program over a pairwise matrix plus a deterministic self-consistency term, and compare against an exact Kemeny–Young solution.

Judge bias and aggregation.. Positional bias in LLM judging is documented by Zheng et al. [10] and Wang et al. [11]. Permutation self-consistency [12] reduces it by averaging over shuffled orderings; we adopt its pairwise

form. Yang et al. [13] compare voting rules (including Borda, Copeland and Kemeny–Young) for aggregating LLM peer evaluations into *model rankings* and report that Kemeny–Young aligns best with human rankings; their setting is leaderboard construction, not answer selection, so we use it as motivation for the Kemeny reference, not as evidence about selection accuracy.

Why more judges or agreement do not necessarily help.. Kohli [5] show that a panel of nine judges carries about two effective votes because the models err on the same items, and that established aggregation methods close at most 11% of the resulting gap. Kim et al. [6] find, across more than 350 LLMs, that larger and more accurate models make more correlated errors. Ding [14] report on 265,000 samples from GPQA Diamond and AIME that agreement is a weak correctness signal (rank correlation 0.20–0.59), weakest for the most self-consistent frontier model. Landesberg [3] show that a judge with moderate global correlation can recover only about a fifth of the improvement perfect selection would give, that pointwise scoring produces ties in two thirds of comparisons, and that explicit pairwise judging recovers markedly more. Our recovery metric (Section 4.5) follows their recommendation to report within-prompt selection quality rather than global agreement.

Rank aggregation.. The Kemeny–Young rule [15, 16] is the unique neutral, consistent, Condorcet-consistent ranking rule [16]; computing it exactly is NP-hard [17], and the rank-aggregation literature [18] motivates tractable surrogates. Coppersmith et al. [4] prove that ordering by weighted number of wins is a 5-approximation for weighted tournaments, the result Proposition 3.1 uses.

3. Method

3.1. Pipeline

Given a query, the pipeline runs five stages (Figure 1).

Generate. Sample N candidate answers, either from one model with N distinct seeds or round-robined across several models for heterogeneity. Each candidate must satisfy a domain-specific validity predicate (for example, an extractable answer letter on GPQA): a non-empty completion can still be unusable. On failure the slot is resampled with a freshly varied seed, up to three times; a problem whose slot never yields a valid answer is reported as ungraded rather than scored on invalid text.

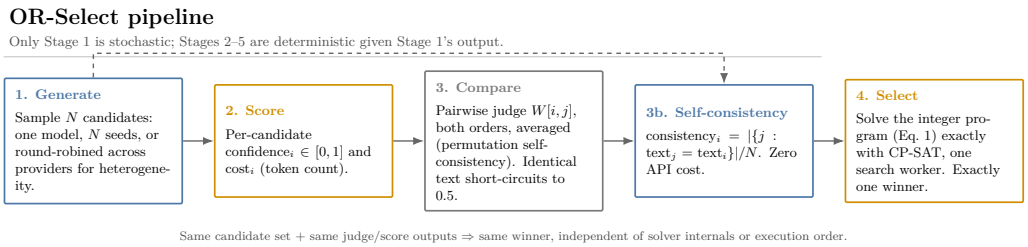


Figure 1: The OR-Select pipeline. Generation (Stage 1) is the only stochastic stage; every later stage is deterministic given Stage 1's output and the judge's outputs. Stage 3b runs in parallel with Stage 3, from the same candidate texts, at zero additional API cost.

Score. Obtain a per-candidate confidence $\text{confidence}_i \in [0,1]$ and a cost proxy cost_i, the candidate's length in whitespace-separated words.

Compare. Build a pairwise preference matrix $W[i,j]$: for each unordered pair an LLM judge is asked twice, once per text order, and the two votes averaged, following permutation self-consistency [12] to cancel positional bias [10, 11] (Figure 2). The reverse entry is $W[j,i] = 1 - W[i,j]$, never re-judged. Pairs with byte-identical candidate text skip the judge and receive $W[i,j] = 0.5$ (Section 6.2 explains why this matters beyond efficiency). Judge calls scale as $N(N-1)$ in the worst case, the dominant cost of the pipeline.

Self-consistency. Compute $\text{consistency}_i = |\{j : \text{text}_j = \text{text}_i\}|/N$ by string comparison: zero API calls, and self-consistency voting [1] as one signal rather than the whole decision rule.

Select. Solve the integer program below exactly with CP-SAT [19].

3.2. Formal Model

Let $x_i \in \{0,1\}$ indicate whether candidate i is selected.

$$\max \sum_i x_i \left(w_1 \cdot \text{quality}_i + w_2 \cdot \text{confidence}_i + w_4 \cdot \text{consistency}_i - w_3 \cdot \text{cost}_i\right) \tag{1}$$

$$\text{s.t.} \quad \sum_i x_i = 1 \tag{2}$$

$$\text{quality}_i \geq \text{min_quality} \quad \text{(optional)} \tag{3}$$

$$x_i \in \{0,1\} \tag{4}$$

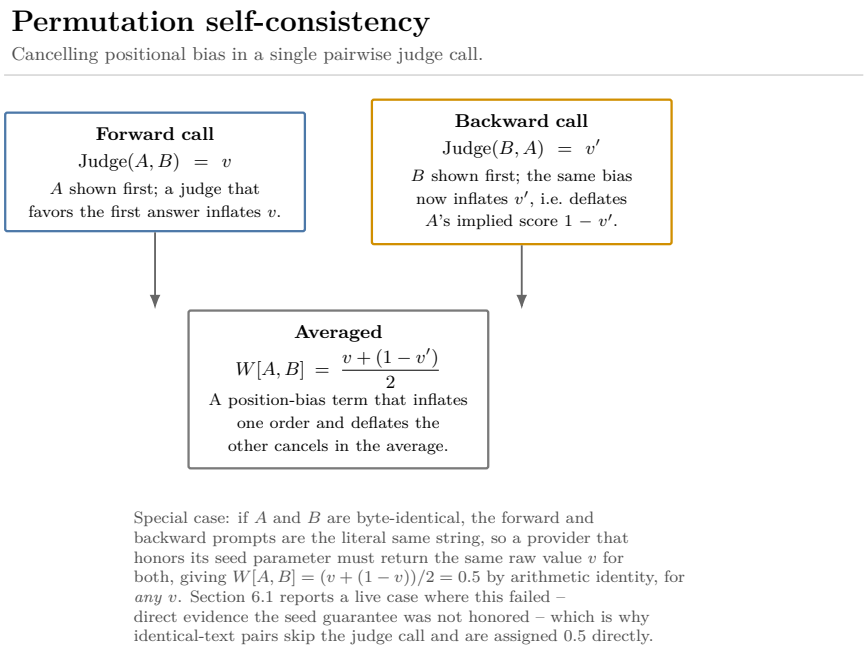


Figure 2: Permutation self-consistency cancels positional bias by averaging the forward and (complement-transformed) backward judge calls for each pair. Byte-identical candidates force $W = 0.5$ by arithmetic identity, independent of the judge's output.

$\text{quality}_i = \text{mean}_j\, W[i,j]$ is the Copeland-style aggregate: the mean weight by which candidate i beats every other candidate. Candidates failing the validity predicate are excluded by a hard gate unless every candidate fails. The default weights $(w_1, w_2, w_4, w_3) = (1.0, 0.2, 0.3, 0.01)$ were set by inspection before any ground-truth run and not tuned. Note the units: quality_i and confidence_i lie in $[0,1]$, whereas cost_i is a word count, so $w_3 = 0.01$ charges one full quality unit per hundred words of difference between candidates. Section 5.5 shows what this does on reasoning-length outputs.

Replay.. Each result row stores the candidate texts, the judge's confidence scores and the full pairwise matrix. The selection step can therefore be rerun exactly under any weights without API calls (Section 5.5); the shipped objective reproduces 480 of 480 stored winners.

Worked example.. Table 1 and Figure 3 decompose Equation (1) for one real open-ended run ($N = 5$, no exact-text duplicates). quality_i is recovered by inverting the saved score_i against the known confidence, cost and consistency, and the recovered argmax is checked by script against the saved winner.

Table 1: Component scores for one real open-ended run (winner marked *). quality_i is recovered by inversion, not stored directly.

Candidate	quality_i	confidence_i	consistency_i	cost_i	score_i
c0	0.606	1.00	0.20	38	0.486
c1	0.387	1.00	0.20	32	0.327
c2*	0.644	1.00	0.20	33	0.574
c3	0.575	1.00	0.20	38	0.455
c4	0.288	1.00	0.20	33	0.218

3.3. Exact Kemeny–Young Baseline

Selecting a winner by quality_i does not require the full ranking, but the Kemeny–Young rule [15, 16] produces one, and computing it exactly gives a reference for the Copeland approximation. For distinct $i \neq j$ let $y_{ij} \in \{0,1\}$ indicate that i is ranked above j. The exact ranking solves

$$\max \sum_{i \neq j} W[i,j]\, y_{ij} \tag{5}$$

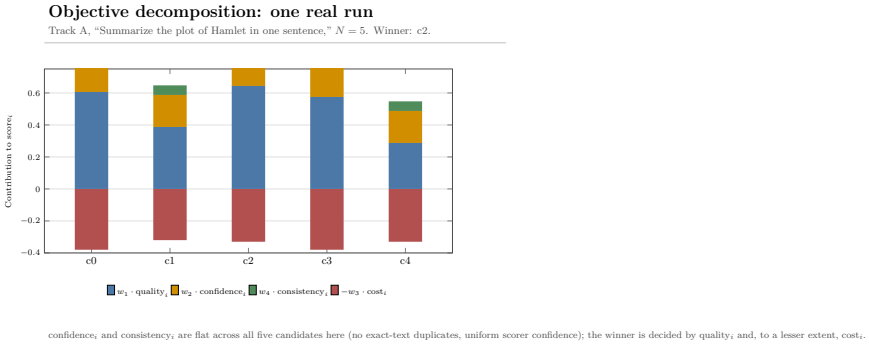


Figure 3: The same run as Table 1, as stacked weighted contributions to score_i. Confidence and consistency are constant across candidates here (no duplicates; uniform scorer confidence), so the ranking is decided by quality and cost. These candidates are one-sentence summaries; on multi-hundred-word reasoning outputs the cost term is far larger relative to quality (Section 5.5).

$$\text{s.t.} \quad y_{ij} + y_{ji} = 1 \quad \forall\, i \neq j \tag{6}$$

$$y_{ij} + y_{jk} + y_{ki} \leq 2 \quad \forall \text{ distinct } i, j, k \tag{7}$$

$$y_{ij} \in \{0, 1\} \tag{8}$$

Constraint (6) forces a total order on every pair; (7) eliminates 3-cycles, which is necessary and sufficient for a linear order. Objective (5) maximizes agreement with the judge's pairwise weights, equivalently minimizing total weighted pairwise disagreement, the quantity Proposition 3.1 bounds. Since $y_{ji} = 1 - y_{ij}$, the implementation (`or_select/kemeny.py`) allocates one binary variable per unordered pair ($\binom{N}{2}$ variables, $O(N^3)$ transitivity constraints). The problem is NP-hard in general [17]; at $N \leq 6$ CP-SAT solves it exactly.

Exact solve time is not uniform in N: on synthetic instances a clear quality gradient solves at $N = 20$ in tens of milliseconds, while near-uniform weights close to 0.5 can leave the search unfinished after 60 seconds. This does not affect the production selection path (a linear score over N candidates). The reference solver runs with a fixed budget of CP-SAT conflicts rather than wall-clock time, so that "same inputs $\Rightarrow$ same ranking" holds even when the budget is exhausted; the solver then returns the best ranking found with an explicit non-optimality flag.

3.4. Approximation Guarantee

Ranking by quality_i is exactly the weighted-out-degree ("weighted number of wins") rule studied by Coppersmith et al. [4].

Proposition 3.1 (4). *Let $W[i,j]+W[j,i] = 1$ for all pairs $i \neq j$. Then ranking candidates by non-increasing $\text{quality}_i = \text{mean}_j W[i,j]$ is a 5-approximation to the exact Kemeny-optimal ranking, measured by total weighted pairwise disagreement, and the analysis is tight.*

The probability constraint holds by construction ($W[j,i] := 1 - W[i,j]$), so the proposition applies without further assumptions. It bounds the disagreement of the *full ranking*; it does not bound whether the top-ranked candidate equals the exact Kemeny winner, and the shipped objective adds confidence, consistency and cost terms to quality_i. We therefore measure the top-1 agreement empirically (Section 5.6). Given fixed inputs the solve is exact: CP-SAT with a single search worker returns the same optimum regardless of machine or call order. This is true by construction; what we measure is whether the *inputs* to the solve are stable and informative.

4. Experimental Protocol

4.1. Setup

Unless noted otherwise, generation, scoring and judging use one free-tier hosted model, NVIDIA Nemotron-3.5-Lightning (`nvidia/nemotron-3.5-lightning:free` on OpenRouter). Judge and confidence calls run with reasoning disabled and a 256-token cap; a cut-off reply raises an error rather than being parsed. Generation is capped at 32,768 output tokens; a cut-off draw is resampled with fresh seeds, and a problem whose slot never completes is recorded as ungraded. Ungraded problems are retried in a flagged second pass at 65,536 tokens (clamped to each model's ceiling); we report accuracy with and without the rescued problems (Table 9).

Heterogeneous generation. $N = 6$ candidates are round-robined over three vendors' models: Nemotron-3.5-Lightning (NVIDIA), Laguna-S 2.1 (Poolside) and North-Mini-Code (Cohere), two candidates per model, with the judge held fixed on Nemotron. Laguna-S 2.1 was withdrawn by the provider while the study ran. Problems graded before that keep their Laguna candidates; for problems generated afterwards its slot is filled by Xing4.0-29B (Xingchen; 26 candidates in total). The tables report how many graded problems still contain Laguna candidates (Table 8).

Homogeneous generation. $N = 3$ candidates from Nemotron with distinct seeds.

4.2. Baselines

Self-consistency: plurality vote over extracted final answers [1], with ties broken by candidate order. **Greedy single-sample**: the first of the N candidates, i.e. no ensembling. **Exact Kemeny–Young**: the winner of the exact ranking of Section 3.3, computed on the same pairwise matrix.

4.3. Judge validation: RewardBench

Following Landesberg's caution that judge calibration does not predict selection accuracy, we first validate the judge in isolation on RewardBench [20]: for each (prompt, chosen, rejected) triple, the identical forward+backward judge code path used in the pipeline must prefer `chosen`. The sample is $N = 161$, the first 7 rows of each of 23 subsets (equal per-subset weighting, matching RewardBench's scoring), pre-registered before results were seen. The pre-registration treated accuracy in the range reported for capable general-purpose judges (roughly 70–90%) as support for trusting the judge's quality signal.

4.4. Evaluation tracks

B – GSM8K [21]: 40 test problems, $N = 3$ homogeneous and $N = 6$ heterogeneous; the answer is the last number in the response, because reasoning models restate numbers from the problem before the final answer. **C – MATH Level 5**: the 134 Level-5 problems of MATH-500 [22, 23], exact-match on the `\boxed{}` answer. **D – AIME 2026**: 30 problems [24]. **E – GPQA Diamond**: 198 graduate-level multiple-choice questions [25], exact-letter grading. Tracks C–E are chosen because they are not saturated for the model used [6, 14]. Track A (three open-ended queries without ground truth) is reported only as a pilot illustration (Section 5.8).

4.5. Metrics

Coverage. Every table reports graded/total, where total is the benchmark size, never the number of rows attempted; ungraded problems are not silently dropped (Table 8). *Paired tests.* For two selectors on the same graded problems we count discordant problems and report the exact two-sided McNemar (binomial) test, the test we pre-registered; pooled tests sum discordant counts over tracks. *Headroom and recovery.* For each problem we score every

candidate. The *floor* is the mean candidate accuracy (a random pick), the *oracle* is the fraction of problems with at least one correct candidate, and a selector's *recovery* is (acc − floor)/(oracle − floor) [cf. 3]. Recovery shows how much of what could be gained a selector captures; negative values mean it does worse than a random pick. *Offline ablations* (Section 5.5) replay the selection step on stored judge outputs; they are post hoc and exploratory.

5. Results

5.1. Judge validation: RewardBench

Overall accuracy is 60.5% (mean of category means, $N = 161$, micro-average 60.9%, 95% Wilson interval for the micro-average [53.2%, 68.1%]), below the range the pre-registration treated as sufficient to trust the quality signal. Table 2 gives the breakdown. Reasoning pairs, the category most relevant to Tracks B–E, reach 73.5% (36/49); Chat Hard, which is adversarial, is at 42.9%, below the 50% chance level. When the judge is rotated across the three heterogeneous-pool models instead of being held fixed, overall accuracy is 48.5% (N=161), indistinguishable from guessing; the pipeline therefore keeps the judge fixed.

A length diagnostic on the same rows splits by whether the human-preferred response is the longer one: accuracy is 68.4% (54/79, 95% CI [57.5%, 77.6%]) when it is, and 53.7% (44/82, [42.9%, 64.0%]) when it is shorter; the intervals overlap, so this is a direction to watch rather than a proven bias.

An earlier pilot, run on 2026-08-22 with a larger model (Nemotron-3-Ultra, since withdrawn by the provider) and a reasoning-enabled judge call, reported a higher figure (90.1%). The judge call was changed to its reasoning-disabled, 256-token form on 2026-10-07 because the reasoning-enabled calls were frequently cut off at the token cap (six of ten in a spot check). That pilot used a different model and judge configuration from every result below, and we do not rely on it.

5.2. Accuracy against the baselines

Table 3 and Figure 4 give accuracy on graded problems. With homogeneous generation (rows B, C, D, E) the three non-exact selectors are within a few problems of each other on every track. On GSM8K all candidates agree on all 40 problems, so no selector can differ; on MATH Level 5 self-consistency is ahead of OR-Select by two problems; on AIME OR-Select is ahead by one;

Table 2: `judge_fn` accuracy on RewardBench, by category ($N = 161$).

Category	Rows	Accuracy
Reasoning (math-prm, 6 `hep-*` code subsets)	49	73.5%
Chat	35	68.6%
Chat Hard (adversarial LLMBar subsets)	42	42.9%
Safety	35	57.1%

on GPQA Diamond they tie. Table 4 makes the comparison paired: pooled over the four homogeneous tracks, OR-Select is right where self-consistency is wrong on 7 problems and wrong where it is right on 8 ($p = 1.000$, $n = 399$). There is no evidence of an advantage or a disadvantage.

With heterogeneous generation (rows B′–E′) OR-Select falls behind: on MATH Level 5 it is right where self-consistency is wrong on 0 problems and wrong where it is right on 15 (79.4% versus 90.8%), on AIME on 0 versus 6, and pooled over the four heterogeneous tracks on 2 versus 25 problems ($n = 286$, $p < 0.001$). Heterogeneous coverage of MATH, AIME and GPQA is still partial (Table 8); the direction on the tracks that are (nearly) complete, GSM8K and MATH, is unambiguous.

Table 3: Accuracy (%) on graded problems, by track and selector. '–': no Kemeny–Young comparator (GSM8K). Bold: best of the three practical selectors when they differ. Primed rows: heterogeneous generation.

Track	Graded/Total	Greedy	Self-cons.	OR-Select	Kemeny–Young
B — GSM8K, homog.	40/40	100.0	100.0	100.0	–
B′ — GSM8K, heterog.	40/40	**100.0**	**100.0**	95.0	–
C — MATH L5, homog.	134/134	89.6	**91.0**	89.6	90.3
C′ — MATH L5, heterog.	131/134	89.3	**90.8**	79.4	80.2
D — AIME 2026, homog.	30/30	80.0	86.7	**90.0**	83.3
D′ — AIME 2026, heterog.	25/30	**96.0**	**96.0**	72.0	92.0
E — GPQA Diamond, homog.	195/198	75.4	**79.0**	**79.0**	78.5
E′ — GPQA Diamond, heterog.	90/198	76.7	**83.3**	**83.3**	74.4

5.3. Headroom and recovery

Accuracy alone hides how much a selector could have gained (Table 5). On GSM8K there is none: floor and oracle coincide. On GPQA Diamond with homogeneous generation the oracle is 87.7% against a floor of 77.3%,

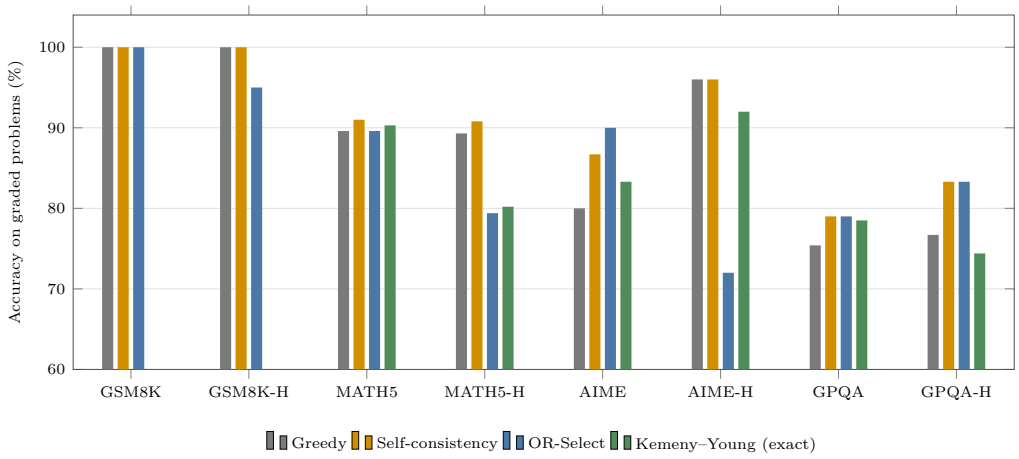


Figure 4: Accuracy on graded problems by track and selector, from Table 3. "-H": heterogeneous generation. Coverage differs across bars (Table 8).

Table 4: Paired comparison of OR-Select with each reference selector. Cells give (problems where OR-Select is right and the reference wrong)/(the reverse) and the exact two-sided McNemar p-value.

Track	vs. SC	p	vs. greedy	p	vs. Kemeny–Young	p
B — GSM8K, homog.	0/0	–	0/0	–	–	–
B′ — GSM8K, heterog.	0/2	0.500	0/2	0.500	–	–
C — MATH L5, homog.	0/2	0.500	1/1	1.000	1/2	1.000
C′ — MATH L5, heterog.	0/15	<0.001	3/16	0.004	7/8	1.000
D — AIME 2026, homog.	1/0	1.000	3/0	0.250	2/0	0.500
D′ — AIME 2026, heterog.	0/6	0.031	0/6	0.031	1/6	0.125
E — GPQA Diamond, homog.	6/6	1.000	13/6	0.167	9/8	1.000
E′ — GPQA Diamond, heterog.	2/2	1.000	8/2	0.109	11/3	0.057

so a perfect selector would add about ten points; both self-consistency and OR-Select capture 16% of that headroom. On heterogeneous MATH Level 5 and AIME, self-consistency recovers most of the headroom while OR-Select scores *below* the random-pick floor. The selector is not merely failing to help; in heterogeneous pools it is anti-informative.

Table 5: Headroom and recovery on graded problems. Floor: mean candidate accuracy. Oracle: some candidate correct. Recovery (%): (acc − floor)/(oracle − floor); negative means below a random pick.

Track	Graded	Floor	SC	OR	Oracle	Rec. SC	Rec. OR
B — GSM8K, homog.	40	99.2	100.0	100.0	100.0	100	100
B′ — GSM8K, heterog.	40	97.5	100.0	95.0	100.0	100	-100
C — MATH L5, homog.	134	89.8	91.0	89.6	91.8	62	-12
C′ — MATH L5, heterog.	131	82.6	90.8	79.4	92.4	84	-32
D — AIME 2026, homog.	30	85.6	86.7	90.0	90.0	25	100
D′ — AIME 2026, heterog.	25	78.7	96.0	72.0	96.0	100	-38
E — GPQA Diamond, homog.	195	77.3	79.0	79.0	87.7	16	16
E′ — GPQA Diamond, heterog.	90	76.3	83.3	83.3	92.2	44	44

5.4. Why heterogeneous selection fails: the judge prefers one generator

Across the heterogeneous problems we know which model produced every candidate and which candidate OR-Select picked (Table 6). With a uniform pool each model would be picked a third of the time. The judge picks the Cohere candidate in 69.7% of problems although Cohere's candidates (87.2% correct) are as accurate as Nemotron's (87.6%). It picks Nemotron, the judge's own model, in only 7.9% of problems, so self-preference is not the explanation. The picked Cohere candidates are right 81.5% of the time, below the 87.2% of Cohere's candidates in general: the preference selects a style, not correctness. The weakest model, Laguna-S (73.3% correct), is picked in 22.0% of problems with 67.9% accuracy. The winner is the longest candidate in only 1% of heterogeneous problems (chance 17%) and 5% of homogeneous problems (chance 33%); on average it has 0.45 (hetero.) and 0.67 (homog.) times the mean candidate length.

5.5. Diagnosing the objective: scale and the judge's own signal

Because every stored row carries the judge's outputs, we replay the selection step under other weights at no API cost (480/480 stored winners are

Table 6: Heterogeneous pools, pooled over the four tracks (286 graded problems). Candidates: all candidates generated by the model and their accuracy (%). Picks: how often OR-Select selected that model's candidate, with share, and the accuracy of those picks (%).

Model	Candidates	Cand. acc.	OR-Select picks	Pick acc.
Nemotron-3.5 (judge model)	572	87.6	19 (7.9%)	100.0
North-Mini-Code	572	87.2	168 (69.7%)	81.5
Laguna-S 2.1	546	73.3	53 (22.0%)	67.9
Xing4.0-29B	26	50.0	1 (0.4%)	0.0

reproduced by the shipped objective, so the replay is faithful). Two facts follow.

The cost term dominates. Within a problem, the spread of the cost term $w_3 \cdot \text{cost}_i$ across candidates has median 0.89 against 0.20 for the quality term, and exceeds it in 444 of 480 replayed problems. On reasoning-length outputs the default objective is, to a first approximation, "pick the shortest valid candidate".

Removing it is not enough in heterogeneous pools. Table 7 replays four variants: the default; no cost term; Copeland quality alone; and plurality vote with Copeland quality as the tie-break. With homogeneous generation all variants are within about one point of self-consistency (the default and no-cost variants win 6 and 7 problems against self-consistency's 5 and 6; $n = 199$). With heterogeneous generation the no-cost and quality-only variants recover only part of the gap (82.2% and 81.9% against self-consistency's 90.0%, $n = 281$; exact McNemar $p < 0.001$ and < 0.001 respectively), and even using the judge only to break ties in a plurality vote does not beat plain voting (wins/losses 0/5). The judge's pairwise signal thus carries no accuracy information beyond voting here, and it is harmful when candidates come from different models. These ablations are post hoc and exploratory; the primary comparison is the pre-registered one in Section 5.2.

5.6. Exact Kemeny–Young reference

Across the 605 graded problems that have an exact Kemeny–Young solution, the shipped selector and the exact winner give different answers on 91. The selector alone is right on 31 and the exact winner alone on 27 problems, so the exact solution is not more accurate either (Table 4, last columns). This is the empirical counterpart of Proposition 3.1: the full-ranking guaran-

Table 7: Offline replay of the selection step on stored judge outputs (accuracy %, problems whose judge outputs were stored). SC: stored self-consistency result on the same problems.

Track	Replayable	SC	Default	No cost	Quality only	SC+quality
B′ GSM8K, heterog.	38/40	100.0	94.7	100.0	100.0	100.0
C MATH L5, homog.	25/134	84.0	84.0	84.0	84.0	84.0
C′ MATH L5, heterog.	130/131	90.8	79.2	83.1	83.1	87.7
D AIME 2026, homog.	9/30	66.7	66.7	66.7	66.7	66.7
D′ AIME 2026, heterog.	24/25	95.8	70.8	75.0	75.0	95.8
E GPQA Diamond, homog.	165/195	77.6	78.2	78.2	78.2	76.4
E′ GPQA Diamond, heterog.	89/91	83.1	83.1	75.3	74.2	82.0

tee does not control the top-1 winner, and in this setting neither aggregation rule helps because both consume the same weak pairwise signal.

5.7. Coverage and the higher-cap second pass

Table 8 reports coverage, including problems that never produced a graded answer (truncated at the token cap after resampling, other failures, or not yet attempted). The second pass at 65,536 tokens rescued the problems in Table 9. These are the problems with the longest reasoning, and they are harder: including them lowers accuracy by roughly one to six points for both selectors, without changing any comparison above.

Table 8: Coverage by track. Truncated: no valid answer after resampling because generation hit the output cap. Other: failed for another reason (including one problem graded under a superseded judge version, left out). Untried: not yet attempted. With Laguna: graded heterogeneous problems that still contain Laguna-S candidates.

Track	Total	Graded	Truncated	Other	Untried	With Laguna
B — GSM8K, homog.	40	40	0	0	0	–
B′ — GSM8K, heterog.	40	40	0	0	0	40
C — MATH L5, homog.	134	134	0	0	0	–
C′ — MATH L5, heterog.	134	131	3	0	0	131
D — AIME 2026, homog.	30	30	0	0	0	–
D′ — AIME 2026, heterog.	30	25	5	0	0	23
E — GPQA Diamond, homog.	198	195	2	1	0	–
E′ — GPQA Diamond, heterog.	198	90	19	3	86	79

5.8. Pilot illustration: open-ended queries

Three open-ended queries ($N = 5$ each, no ground truth) were run in an earlier pilot to check that the pipeline behaves sensibly where self-consistency

Table 9: Effect of the flagged second pass. Main protocol: problems that completed under the 32,768-token cap. Rescued: problems completed only under the higher cap. Final: all graded problems.

Track	Main n	Main OR	Main SC	Rescued	Final OR	Final SC
C — MATH L5, homog.	132	90.9	92.4	2	89.6	91.0
D — AIME 2026, homog.	24	95.8	91.7	6	90.0	86.7
D′ — AIME 2026, heterog.	23	78.3	100.0	2	72.0	96.0
E — GPQA Diamond, homog.	174	82.2	82.8	21	79.0	79.0

has nothing to vote on. Across the 15 candidates, none was byte-identical to another, so self-consistency resorted to an arbitrary tie-break on every query, and OR-Select differed from both baselines on all three (score margins 0.088, 0.446 and a near-tie of 0.003). With three queries and no ground truth this is an illustration, not evidence about quality. Two pilot ablations on the same candidate sets (setting $w_4 = 0$; forward-only judging) changed 0 and 2 of the 3 winners respectively: the self-consistency term was inert because no candidate set clustered, and bidirectional judging mattered in two of three cases, consistent with position bias being a first-order judge failure mode [10, 12, 11]. Three instances cannot characterize when the directions disagree.

6. Discussion

6.1. What the audit shows

The method does what it is designed to do: the selection is exactly reproducible and exactly replayable, with an exact Kemeny–Young reference. It does not do what one might hope: it does not improve on majority vote. Four factors, all measured above, account for this. First, the judge is only moderately reliable and, at 42.9% on adversarial pairs, poor where candidates are close. Second, the headroom is small on the saturated benchmarks (floor $\approx$ oracle on GSM8K, within two points on MATH Level 5), leaving little to select. Third, where headroom exists (GPQA Diamond) the judge's pairwise signal recovers no more of it than a vote does. Fourth, in heterogeneous pools the judge's preference for one generator and the default objective's length-scale mismatch turn a neutral selector into a harmful one. This agrees with the literature on correlated errors [5, 6] and on the weak link between judge metrics and selection accuracy [3], and adds a within-pool, ground-truth measurement of the effect.

Reproducibility is not accuracy. A selector can be bit-for-bit reproducible and systematically wrong; the replay machinery is what made the diagnoses above cheap, and we regard it as a transferable practice for judge-based selectors.

6.2. A concrete non-determinism finding

In an earlier pilot run on a different provider and model, three candidates had byte-identical text. By construction every pairwise weight among identical text must be 0.5, since the forward and backward judge calls are the same prompt and $(v + (1 - v))/2 = 0.5$. The observed quality scores were 0.375, 0.50 and 0.625, direct evidence that the provider's `seed` parameter was not reliably honored in that run. This is a single observation; the exact-match short-circuit (Section 3.1) removes this failure mode for identical text but not for different texts that receive unstable scores, which remains open.

6.3. Retry independence and extraction fallbacks

Each validity-retry draws with a freshly varied seed, so a persistently invalid slot gets three independent samples rather than three repeats of what a seed-honoring backend already returned. Extraction accepts more than one marker: GPQA also recognizes a prose "the answer is X" pattern alongside `ANSWER: X` and `\boxed{X}`, because its answer space is four letters; MATH's open-ended `\boxed{}` extraction has no prose fallback, since a wrong guess there would convert an honest "ungraded" problem into a confidently incorrect one. This asymmetry is a design choice.

6.4. Limitations

One judge model. Generation, scoring and judging use one free-tier model. The RewardBench result shows the judge is only moderately reliable; a stronger judge may change the conclusions, and we cannot say how. Judge diversity [5] was not tested, only generator diversity. The heterogeneous judge-rotation RewardBench result suggests that simply rotating weak judges is not a remedy.

Mixed pool and partial coverage. Heterogeneous problems graded before Laguna-S 2.1 was withdrawn keep its candidates (Table 8), so the heterogeneous rows describe a mixed pool, not a single fixed one. Heterogeneous coverage is incomplete on AIME and GPQA; conclusions rest on the complete tracks and on the direction of the incomplete ones.

Scale and power. Candidate sets are small ($N = 3$–6), and the homogeneous comparison with self-consistency has few discordant problems, so we can exclude large effects but not small ones: a null result at this size is not evidence of equivalence.

Post hoc analyses. The weight ablations, selection-share and length analyses were not pre-registered. The default weights were fixed before the ground-truth runs and not tuned afterwards; we deliberately report the shipped objective as the primary result, and the cost-scale problem as a finding about it.

Benchmark scope. Tracks are mathematics and science multiple choice or exact-match; open-ended quality, where self-consistency is undefined and a judge is unavoidable, is covered only by a three-query pilot.

7. Conclusion

We formulated best-of-N answer selection as an exactly reproducible integer program over a pairwise matrix and a deterministic self-consistency term, inherited a 5-approximation guarantee against the NP-hard Kemeny-optimal ranking, and built an audit around it: exact Kemeny–Young reference, paired exact tests, headroom and recovery, coverage accounting, and replayable judge outputs. On four ground-truth benchmarks the audit finds no accuracy benefit of judge-based selection over majority vote with a single homogeneous generator, and a significant harm with heterogeneous generators, traced to a length-scale mismatch in the objective and a generator-specific judge preference rather than to nondeterminism. The practical lessons are to validate the judge on the configuration actually shipped, to report recovery against a floor and an oracle rather than accuracy alone, and to keep judge outputs so that a selector can be audited without new API calls. Whether a stronger or more diverse judge makes judge-based selection pay off is the open question this protocol is built to answer.

Data and code availability

Code, stored judge outputs, generated tables and the pre-registration documents are available at `https://github.com/DressPD/or-select`. All numbers in Section 5 are regenerated from the stored outputs by the scripts named at the top of the source.

References

[1] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, D. Zhou, Self-consistency improves chain of thought reasoning in language models, in: International Conference on Learning Representations (ICLR), 2023. arXiv:2203.11171.

[2] X. Chen, R. Aksitov, U. Alon, J. Ren, K. Xiao, P. Yin, S. Prakash, C. Sutton, X. Wang, D. Zhou, Universal self-consistency for large language model generation, 2023. arXiv:2311.17311.

[3] E. Landesberg, When llm judge scores look good but best-of-n decisions fail, 2026. arXiv:2603.12520.

[4] D. Coppersmith, L. Fleischer, A. Rudra, Ordering by weighted number of wins gives a good ranking for weighted tournaments, ACM Transactions on Algorithms 6 (2010) 55:1–55:13.

[5] G. Kohli, Nine judges, two effective votes: Correlated errors undermine llm evaluation panels, 2026. arXiv:2605.29800.

[6] E. Kim, A. Garg, K. Peng, N. Garg, Correlated errors in large language models, in: International Conference on Machine Learning (ICML), 2025. arXiv:2506.07962.

[7] Y. Liu, H. Zhou, Z. Guo, E. Shareghi, I. Vulić, A. Korhonen, N. Collier, Aligning with human judgement: The role of pairwise preference in large language model evaluators, in: Conference on Language Modeling (COLM), 2024. arXiv:2403.16950.

[8] D. Jiang, X. Ren, B. Y. Lin, Llm-blender: Ensembling large language models with pairwise ranking and generative fusion, in: Proceedings of the Association for Computational Linguistics (ACL), 2023. arXiv:2306.02561.

[9] Z. Chen, Z. Ji, Q. Mao, H. Wu, J. Song, J. Cheng, B. Qin, Z. Li, J. Li, K. Sun, Z. Wang, Y. Ban, Z. Sun, X. Ji, H. Sun, X. Huang, Scoring, reasoning, and selecting the best! ensembling large language models via a peer-review process, 2025. arXiv:2512.23213.

[10] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, I. Stoica, Judging llm-as-a-judge with mt-bench and chatbot arena, in: Advances in Neural Information Processing Systems (NeurIPS), 2023. arXiv:2306.05685.

[11] P. Wang, L. Li, L. Chen, Z. Cai, D. Zhu, B. Lin, Y. Cao, Q. Liu, T. Liu, Z. Sui, Large language models are not fair evaluators, 2023. arXiv:2305.17926.

[12] R. Tang, X. Zhang, X. Ma, J. Lin, F. Ture, Found in the middle: Permutation self-consistency improves listwise ranking in large language models, in: Findings of the Association for Computational Linguistics: NAACL 2024, 2024. arXiv:2310.07712.

[13] G. Yang, Y. Liu, S. Miao, X. Liang, Z. Liu, H. Huang, Llms judge themselves: A game-theoretic framework for human-aligned evaluation, 2025. arXiv:2510.15746.

[14] K. Ding, When llms agree, are they right? auditing self-consistency and cross-model agreement as confidence signals, 2026. arXiv:2607.08065.

[15] J. G. Kemeny, Mathematics without numbers, Daedalus 88 (1959) 577–591.

[16] H. P. Young, A. Levenglick, A consistent extension of Condorcet's election principle, SIAM Journal on Applied Mathematics 35 (1978) 285–300. doi:10.1137/0135023.

[17] J. J. Bartholdi, C. A. Tovey, M. A. Trick, Voting schemes for which it can be difficult to tell who won the election, Social Choice and Welfare 6 (1989) 157–165. doi:10.1007/BF00303169.

[18] C. Dwork, R. Kumar, M. Naor, D. Sivakumar, Rank aggregation methods for the Web, in: Proceedings of the 10th International Conference on World Wide Web (WWW), 2001, pp. 613–622. doi:10.1145/371920.372165.

[19] L. Perron, V. Furnon, OR-Tools CP-SAT solver, Google, https://developers.google.com/optimization/, 2023.

[20] N. Lambert, V. Pyatkin, J. Morrison, L. Miranda, B. Y. Lin, K. Chandu, N. Dziri, S. Kumar, T. Zick, Y. Choi, N. A. Smith, H. Hajishirzi, Rewardbench: Evaluating reward models for language modeling, in: Findings of the Association for Computational Linguistics: NAACL 2025, 2025. arXiv:2403.13787.

[21] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, J. Schulman, Training verifiers to solve math word problems, 2021. arXiv:2110.14168.

[22] D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, J. Steinhardt, Measuring mathematical problem solving with the MATH dataset, in: Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track, 2021. arXiv:2103.03874.

[23] H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, K. Cobbe, Let's verify step by step, 2023. arXiv:2305.20050.

[24] M. Balunović, J. Dekoninck, I. Petrov, N. Jovanović, M. Vechev, MathArena: Evaluating LLMs on uncontaminated math competitions, SRI Lab, ETH Zurich, https://matharena.ai/, 2025.

[25] D. Rein, B. L. Hou, A. C. Stickland, J. Petty, R. Y. Pang, J. Dirani, J. Michael, S. R. Bowman, Gpqa: A graduate-level google-proof q&a benchmark, in: Conference on Language Modeling (COLM), 2024. arXiv:2311.12022.